

Aplikacja mobilna – „Baza połowów”

Cel: zbuduj aplikację z listą połowów, dodawaniem nowych rekordów oraz zapisem danych do pliku JSON.

Dane:

- **Imie** (string)
- **Nazwisko** (string)
- **Lowisko** (string)
- **GatunekRyby** (string)
- **Waga** (double)
- **DataPolowu** (DateTime)

Punkt kontrolny 1 — Lista połowów się wyświetla

1. Utwórz projekt **.NET MAUI App**.
2. W `MainPage.xaml.cs` utwórz klasę `PolowRecord` z właściwościami: `Imie`, `Nazwisko`, `Lowisko`, `GatunekRyby`, `Waga`, `DataPolowu` (na razie zwykłe `{get; set;}`).
3. Utwórz `ObservableCollection<PolowRecord>` `Polowy` i dodaj 3 rekordy startowe, np.:
 - Jan, Kowalski, Wisła, Sandacz, 2.4, `DateTime.Now`
 - Piotr, Nowak, Odra, Szczupak, 4.1, `DateTime.Now`
 - Adam, Wiśniewski, Jezioro Nyskie, Karp, 3.7, `DateTime.Now`
4. Ustaw `BindingContext = this;`
5. W XAML dodaj `CollectionView` z `ItemsSource="{Binding Polowy}"` i pokaż w elemencie listy:
 - Imie + Nazwisko
 - Lowisko
 - GatunekRyby
 - Waga: X kg

✓ **Zaliczenie:** widoczna lista 3 połowów.

Punkt kontrolny 2 — Dodawanie nowego połowu

1. Nad listą dodaj pola `Entry` do wpisania:
 - o Imię
 - o Nazwisko
 - o Lowisko
 - o GatunekRyby
 - o Waga
2. Dodaj przycisk „Dodaj połów”.
3. W `MainPage.xaml.cs` utwórz metodę obsługującą kliknięcie przycisku.
4. W tej metodzie:
 - o pobierz dane z pól `Entry`,
 - o sprawdź, czy wszystkie pola są uzupełnione,
 - o zamień `Waga` z tekstu na liczbę,
 - o utwórz nowy obiekt `PolowRecord`,
 - o ustaw `DataPolowu = DateTime.Now`,
 - o dodaj rekord do kolekcji `Polowy`.
5. Po dodaniu rekordu wyczyść pola formularza.

✓ **Zaliczenie:** po wpisaniu danych i kliknięciu przycisku nowy połów pojawia się na liście.

Punkt kontrolny 3 — Usuwanie wszystkich połowów z listy

1. Pod formularzem albo nad listą dodaj przycisk „Usuń wszystkie rekordy”.
2. W `MainPage.xaml.cs` utwórz metodę obsługującą kliknięcie tego przycisku.
3. W tej metodzie:
 - o wyświetl okno potwierdzenia `DisplayAlert`,
 - o jeśli użytkownik wybierze **Tak**, wyczyść kolekcję `Polowy`,
 - o jeśli użytkownik wybierze **Nie**, nie rób nic.
4. Po wyczyszczeniu lista ma od razu zniknąć z ekranu.

Wskazówka:

Jeśli `CollectionView` jest podłączony do kolekcji `Polowy`, to do usunięcia wszystkich rekordów wystarczy:

```
Polowy.Clear();
```

✓ **Zaliczenie:** po kliknięciu przycisku i potwierdzeniu wszystkie rekordy znikają z listy.

Punkt kontrolny 4 — Zapis wszystkich rekordów do pliku JSON

1. Dodaj do projektu folder **Data**.
2. W folderze **Data** utwórz plik **JsonStore.cs**.
3. W pliku `JsonStore.cs` utwórz klasę `JsonStore`, która będzie odpowiadała za zapis danych do pliku JSON.
4. W klasie `JsonStore` przygotuj:
 - o pole przechowujące ścieżkę do pliku `polowy.json`,
 - o konstruktor ustawiający tę ścieżkę,
 - o metodę `SaveAllAsync(List<PolowRecord> records)`, która zapisze całą listę rekordów do pliku.
5. W `MainPage.xaml.cs` utwórz obiekt `JsonStore`.
6. W metodzie dodającej nowy połów, po dodaniu rekordu do kolekcji `Polowy`, wywołaj zapis całej kolekcji do pliku JSON.
7. Na tym etapie aplikacja ma **zapisywać dane do pliku**, ale jeszcze nie musi ich odczytywać po ponownym uruchomieniu.

Wskazówka:

W pliku `JsonStore.cs` przydadzą się:

```
using System.Text.Json;
```

oraz konstruktor i metoda podobne do:

```
private readonly string _filePath;

public JsonStore()
{
    _filePath = Path.Combine(FileSystem.AppDataDirectory, "polowy.json");
}

public async Task SaveAllAsync(List<PolowRecord> records)
{
    using var stream = File.Create(_filePath);
    await JsonSerializer.SerializeAsync(stream, records);
}
```

Po dodaniu rekordu do kolekcji możesz wywołać zapis w stylu:

```
await _store.SaveAllAsync(Polowy.ToList());
```

✔ **Zaliczenie:** po dodaniu nowego połowu rekord pojawia się na liście i cała kolekcja zapisuje się do pliku `polowy.json`.